

```
# HIDE OUTPUT
```

```
!git clone https://github.com/NVlabs/stylegan3.git
!pip install ninja
```

```
Cloning into 'stylegan3'...
remote: Enumerating objects: 212, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 212 (delta 0), reused 0 (delta 0), pack-reused 208 (from 2)
Receiving objects: 100% (212/212), 4.17 MiB | 8.00 MiB/s, done.
Resolving deltas: 100% (98/98), done.
Collecting ninja
  Downloading ninja-1.11.1.3-py3-none-manylinux_2_12_x86_64.manylinux2010_x86_64.whl.metadata (5.3 kB)
  Downloading ninja-1.11.1.3-py3-none-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (422 kB)
    422.9/422.9 kB 8.7 MB/s eta 0:00:00
Installing collected packages: ninja
Successfully installed ninja-1.11.1.3
```

```
!ls /content/stylegan3
```

```
avg_spectra.py  dnnlib      environment.yml  gui_utils      metrics        training        viz
calc_metrics.py Dockerfile     gen_images.py   legacy.py      README.md      train.py
dataset_tool.py docs          gen_video.py    LICENSE.txt    torch_utils    visualizer.py
```

```
!ls /content/results
```

```
import sys
sys.path.insert(0, "/content/stylegan3")
import pickle
import os
import numpy as np
import PIL.Image
from IPython.display import Image
import matplotlib.pyplot as plt
import IPython.display
import torch
import dnnlib
import legacy

def seed2vec(G, seed):
    return np.random.RandomState(seed).randn(1, G.z_dim)

def display_image(image):
    plt.axis('off')
    plt.imshow(image)
    plt.show()

def generate_image(G, z, truncation_psi):
    # Render images for dlatents initialized from random seeds.
    Gs_kwargs = {
        'output_transform': dict(func=tflib.convert_images_to_uint8,
                                nchw_to_nhw=True),
        'randomize_noise': False
    }
    if truncation_psi is not None:
        Gs_kwargs['truncation_psi'] = truncation_psi

    label = np.zeros([1] + G.input_shapes[1][1:])
    # [minibatch, height, width, channel]
    images = G.run(z, label, **Gs_kwargs)
    return images[0]

def get_label(G, device, class_idx):
    label = torch.zeros([1, G.c_dim], device=device)
    if G.c_dim != 0:
        if class_idx is None:
            ctx.fail("Must specify class label with --class when using "\
                    "a conditional network")
        label[:, class_idx] = 1
    else:
        if class_idx is not None:
            print("warn: --class=lbl ignored when running on "\
                  "an unconditional network")
    return label

def generate_image(device, G, z, truncation_psi=1.0, noise_mode='const',
                  class_idx=None):
    z = torch.from_numpy(z).to(device)
    label = get_label(G, device, class_idx)
    img = G(z, label, truncation_psi=truncation_psi, noise_mode=noise_mode)
    img = (img.permute(0, 2, 3, 1) * 127.5 + 128).clamp(0, 255).to(\
```

```

    torch.uint8)
    return PIL.Image.fromarray(img[0].cpu().numpy(), 'RGB')

#URL = "https://github.com/jeffheaton/pretrained-gan-fish/releases/"\
#  "download/1.0.0/fish-gan-2020-12-09.pkl"
#URL = "https://github.com/jeffheaton/pretrained-merry-gan-mas/releases/"\
#  "download/v1/christmas-gan-2020-12-03.pkl"
URL = "https://api.ngc.nvidia.com/v2/models/nvidia/research/stylegan3/"\
      "versions/1/files/stylegan3-r-ffhq-1024x1024.pkl"

print(f'Loading networks from "{URL}"...')
device = torch.device('cuda')
with dnnlib.util.open_url(URL) as f:
    G = legacy.load_network_pkl(f)['G_ema'].to(device) # type: ignore

```

➦ Loading networks from "<https://api.ngc.nvidia.com/v2/models/nvidia/research/stylegan3/versions/1/files/stylegan3-r-ffhq-1024x1024.pkl>"

```

# Choose your own starting and ending seed.
SEED_FROM = 1000
SEED_TO = 1003

```

```

# Generate the images for the seeds.
for i in range(SEED_FROM, SEED_TO):
    print(f"Seed {i}")
    z = seed2vec(G, i)
    img = generate_image(device, G, z)
    display_image(img)

```

Seed 1000
Setting up PyTorch plugin "bias_act_plugin"... /usr/local/lib/python3.11/dist-packages/torch/utils/cpp_extension.py:1964: UserWarning: If this is not desired, please set os.environ['TORCH_CUDA_ARCH_LIST'].
warnings.warn(
Done.
Setting up PyTorch plugin "filtered_lrelu_plugin"... /usr/local/lib/python3.11/dist-packages/torch/utils/cpp_extension.py:1964: UserWarning: If this is not desired, please set os.environ['TORCH_CUDA_ARCH_LIST'].
warnings.warn(
Done.



Seed 1001

